CAPSTONE PROJECT

AI Agent System

Skill Set Go EduTech · AI/ML Track · Week 4

17	2	$0
tools across 3 categories	Groq models with fallback	cost - free tools only

1. Overview

This capstone is a general-purpose AI agent built on LangGraph that combines retrieval-augmented generation (RAG) for internship questions, 15 additional tools for calculation, conversion, and live information lookup, and multi-step reasoning to decide which tool (or tools) to call before answering. It ships as both a command-line script (`agent.py`) and a Streamlit chat web app (`app.py`).

2. Architecture

The agent is a loop between an **agent** node (the LLM with 17 tools bound to it) and a **tools** node (executes whichever tool or tools the LLM requested — several can be called independently in one turn, e.g. a unit conversion and a currency conversion together). Control returns to the agent node after each tool call so it can decide whether it has enough information to answer or needs another tool.

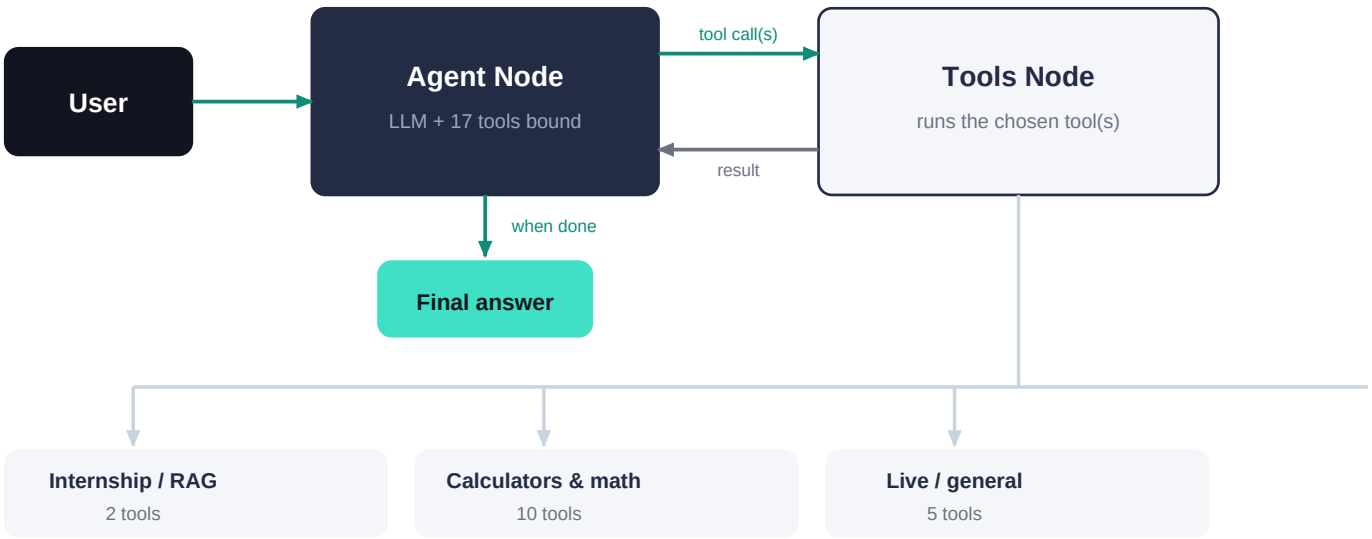

Figure 1 — The agent ↔ tools loop, with tools grouped into three categories.

3. The 17 Tools

Tools are organized into three categories, matching how the system prompt routes each type of question to the right tool.

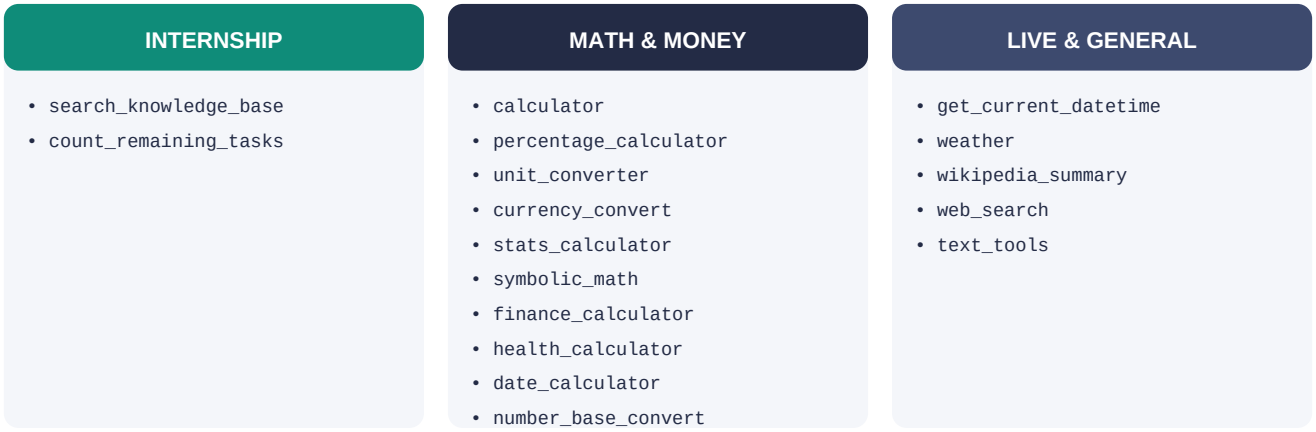

Figure 2 — All 17 tools grouped by category.

Tool	Purpose
search_knowledge_base	RAG retrieval over the internship FAQ via a local FAISS index
count_remaining_tasks	Deterministic progress tracking against the 17-task total
calculator / percentage_calculator	Exact arithmetic and percentage math (discount, tax, % change)
unit_converter / currency_convert	Length, weight, area (incl. marla/kanal), temperature, data, speed, time; live currency rates
stats_calculator / symbolic_math	Mean/median/mode/std-dev; algebra and calculus via SymPy (solve, factor, differentiate, integrate)
finance_calculator / health_calculator	EMI, simple/compound interest; BMI with category
date_calculator / number_base_convert	Date differences, age, weekday, date arithmetic; binary/hex/octal conversion
get_current_datetime / weather	Current time in any IANA timezone; live weather for any city
wikipedia_summary / web_search	Quick factual summaries; live web search for news, prices, and recent events
text_tools	Word/character count, case conversion, reversal, password generation

4. Reliability: Model Fallback

The agent tries two Groq models in order. If a call to the default model fails or times out, it automatically retries with the next model, so a single provider hiccup does not break the conversation.

If a model call fails or times out, the app automatically retries with the next model in line.

Figure 3 — The two-model fallback chain used by both agent.py and the Streamlit app.

5. Multi-Step Workflow (Task 4.2 pattern)

The same conditional-branching pattern from task 4.2 — classify, draft, review, then revise or finalize — underlies how the capstone agent evaluates whether a tool result is sufficient before answering, generalized here into an open-ended agent ↔ tools loop rather than a fixed 3-step sequence.

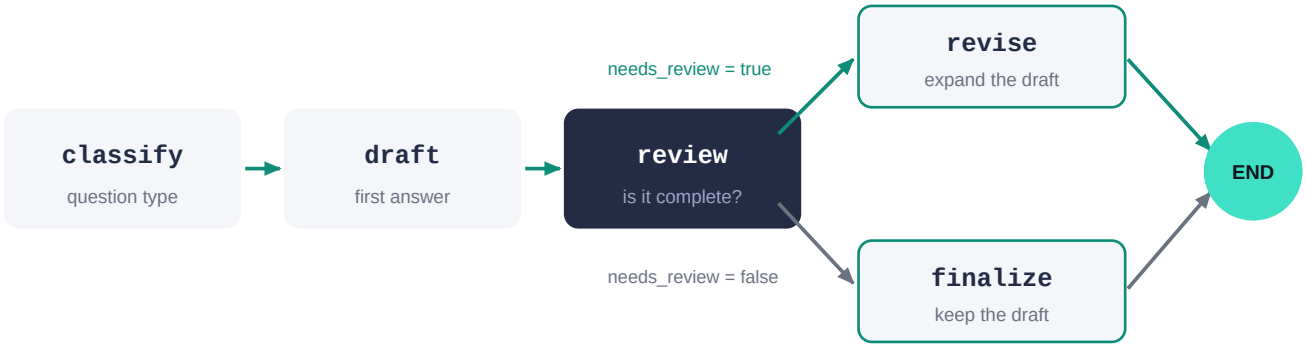

Figure 4 — The conditional-branch pattern the capstone loop generalizes.

6. Streamlit Web App

app.py wraps the same agent in a chat interface: a sidebar lets the user pick the model, adjust creativity (temperature), toggle agent steps, browse chat history, and try 13 example prompts spanning internship questions, conversions, math, and live lookups. Turn on "Show agent steps" to see each tool call and its result under the answer, so the reasoning stays visible rather than hidden.

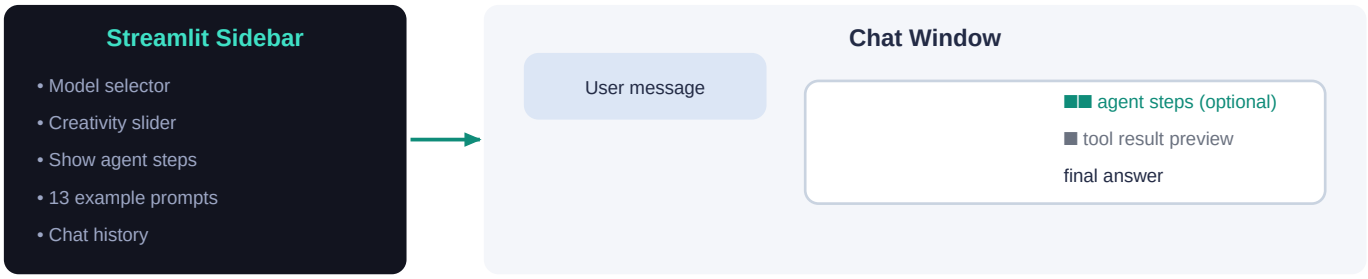

Figure 5 — Sidebar controls and on-demand agent steps in the chat UI.

7. Example Interaction

"Convert 5 marla to square feet and 100 USD to PKR" triggers two independent tool calls in the same turn — unit_converter for the area conversion and currency_convert for the live exchange rate — and the agent combines both results into a single reply, without needing two separate questions.

8. Why This Design

- **Grounded, not guessed:** RAG ties internship answers to actual FAQ content instead of model memory.
- **Precise, not approximate:** every numeric domain (math, currency, dates, units) has its own exact tool instead of relying on the LLM's arithmetic.
- **Resilient:** two-model fallback means a single provider failure does not stop the conversation.
- **Transparent:** the Streamlit UI can show each tool call on request, so the reasoning is visible, not a black box.
- **Scales to N steps and N tools:** the agent → tools → agent loop handles one call or several independent calls per turn equally well.

9. Requirements and Setup — Completely Free

This project uses **zero paid services**. The language models run on Groq's free API (no credit card required) and RAG embeddings run locally via `sentence-transformers/all-MiniLM-L6-v2`. Install with `pip install -r requirements.txt`, set `GROQ_API_KEY` (free at console.groq.com/keys) in a `.env` file, then run `python agent.py` for the command-line version or `streamlit run app.py` for the web app.

> **Live demo:** [add your recorded walkthrough link here] — record a short screen capture of the Streamlit app answering a few example questions live (Loom or OBS both work).

10. Conclusion

This capstone demonstrates the full arc of the track: from Python and machine-learning fundamentals in Week 1, through deep-learning frameworks in Weeks 2–3, to a production-style agentic system in Week 4 that combines retrieval, a broad tool library, multi-model reliability, and a real chat interface — built here entirely on free tools.